

FORGESIGHT AI

Ultimate Master Educational & Viva Handbook

Authors: Mr. Shubham Utekar & Dr. Santosh Rane

Topic: Explainable Federated Digital Twin-Based Predictive Maintenance

Stack: FastAPI, XGBoost, SHAP, Flower, React 18, Three.js, WebGL

Exhaustive Defense & Viva Preparation Guide

Complete breakdown of Frontend, Backend, Every Page, Mathematical Derivations, and 50 Master Q&A.;

CHAPTER 1 — Executive Summary & Core Concepts

Predictive Maintenance (PdM): Analyzes sensor streams (temperature, pressure, vibration, speeds) to project an asset's Remaining Useful Life (RUL), scheduling repairs before breakdowns occur.

The Three Industry Barriers: (1) Data confidentiality prevents centralizing raw logs across plants; (2) High-accuracy ML models function as uninterpretable black boxes; (3) 2D static dashboards are detached from physical 3D machine geometries.

The ForgeSight AI Solution: Jointly integrates Federated Learning (FedBagging), Rényi Differential Privacy (RDP), local on-device TreeSHAP explanations, Split Conformal Prediction intervals, and a 3D WebGL Digital Twin.

Target Capping Formula:

$$y_i = \min(RUL_{raw}, 125)$$

CHAPTER 2 — Architecture & Components Breakdown

Backend Services (FastAPI):

- **ml_engine.py:** Thread-safe Singleton Model Registry for loading joblib checkpoints without memory leaks.
- **xai_engine.py:** Calculates TreeSHAP feature attributions and L-BFGS-B counterfactual optimizations.
- **streaming.py:** Asynchronous WebSocket route broadcasting predictions, bounds, and health matrices.

Frontend Pages (React 18 & Three.js):

- **1. Overview Dashboard:** Metric cards, RUL trend charts, anomaly banners, and health summaries.
- **2. 3D Digital Twin Visualizer:** Renders interactive 3D WebGL model at 60 FPS using React-Three-Fiber. Dynamic materials update colors (Green > Yellow > Red) based on health scores.
- **3. Diagnostics & XAI Page:** Displays SHAP waterfall attribution charts and interactive counterfactual sliders.
- **4. Model Benchmarks Page:** Comparative analytics (XGBoost vs RF vs GBDT vs LR).

CHAPTER 3 — Mathematical Formulations

1. XGBoost Regularized Split Gain:

$$\text{Gain} = 0.5 * [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda) - (G_L + G_R)^2 / (H_L + H_R + \lambda)] - \gamma$$

2. Federated Bagging Aggregation (FedBagging):

$$\text{Tree}_{Global} = \text{Tree}_t \cup (\text{Union}_{\{k=1\}^K} \text{Tree}_{\{k, t+1\}})$$

3. Rényi Differential Privacy (RDP) Budget:

$$\text{eps_rdp} = (T * \alpha) / (\sigma^2) + \ln(1.25 / \delta) / (\alpha - 1) \implies \text{eps} = 1.20 \text{ for } \delta = 10^{-5}$$

4. Split Conformal Boundary Quantile:

$$q_{\hat{}} = \text{Quantile}(\{|y_i - \hat{f}(x_i)|\}, \text{ceil}((n+1)(1-\alpha)) / n) \implies q_{\hat{}} = 5.01 \text{ cycles}$$

5. Counterfactual Optimization Objective:

$$J(\delta) = ||\delta||_1 + \lambda * (\hat{f}(x + \delta) - y_{\text{target}})^2$$

CHAPTER 4 — Master Viva Question Bank (50 Questions & Answers)

Q1: What is the primary objective of ForgeSight AI?

A: To provide an end-to-end, privacy-preserving predictive maintenance framework that estimates Remaining Useful Life (RUL), generates local on-device explanations, bounds predictions with conformal uncertainty intervals, and synchronizes diagnostics with a 3D WebGL Digital Twin.

Q2: What dataset was used to validate the system?

A: The NASA C-MAPSS FD001 benchmark dataset, consisting of 100 training engines (run-to-failure) and 100 test engines (right-censored).

Q3: Why is the Remaining Useful Life (RUL) target capped at 125 cycles?

A: Early operating cycles represent healthy baseline states without degradation. Capping target values at 125 cycles prevents regressors from fitting arbitrary linear trends during healthy phases, focusing model capacity on active wear regimes.

Q4: What is the difference between Reactive, Preventive, and Predictive Maintenance?

A: Reactive repairs equipment after failure (expensive, unsafe). Preventive replaces parts on fixed schedules (wastes healthy components). Predictive uses real-time sensor streams and ML models to schedule maintenance right before failure occurs.

Q5: What are the three primary barriers to industrial AI adoption addressed by this work?

A: (1) Data confidentiality concerns across plant boundaries, (2) The opaque 'black box' nature of ML models, and (3) Isolated 2D tabular dashboards detached from physical machine geometry.

Q6: Why expand 14 raw sensor channels into a 270-dimensional feature space?

A: Raw static telemetry snapshot readings do not capture temporal mechanical wear. Generating rolling statistics (mean, std, min, max), lag states, EMAs, and rates of change provides the temporal context necessary to track degradation.

Q7: Why select XGBoost over Random Forest when Random Forest has a lower MAE (2.879 vs 2.911)?

A: XGBoost achieves a higher R^2 score (0.957 vs 0.954), lower RMSE (4.984 vs 5.153), reduces offline training time by 88% (7.2s vs 60.8s), has a smaller memory footprint, and supports faster edge inference (0.021 ms).

Q8: What is the impact of removing the cycle counter feature during ablation testing?

A: Removing the cycle counter increases MAE from 2.930 to 8.296 cycles (a 183.2% error increase), demonstrating that elapsed operational time is the single strongest indicator of mechanical wear.

Q9: How sensitive is the model to progressive sensor drift?

A: A 10% linear sensor drift increases prediction MAE by 733% (24.249 cycles) because static feature scaling shifts values outside the training distribution, highlighting the need for adaptive standardization.

Q10: What is the average single-sample inference latency of XGBoost?

A: Approximately 0.021 ms per sample on an AMD Ryzen 7 5800H processor, making it well-suited for high-frequency industrial telemetry streaming.

Q11: Why can't standard FedAvg be used for federated XGBoost?

A: Decision tree structures differ across clients based on local split selections, so node parameters cannot be averaged arithmetically. Instead, we use FedBagging to aggregate local tree structures into a global ensemble.

Q12: How is local client privacy protected during federated updates?

A: Clients train local models on private data splits and transmit only tree structures and split parameters to the server. No raw telemetry records are ever transmitted.

Q13: What privacy framework is used, and what is the final privacy budget?

A: Rényi Differential Privacy (RDP) with noise scale $\sigma = 0.05$ and clipping norm $C = 1.0$. Across 5 communication rounds, this yields a cumulative privacy budget of $\epsilon = 1.20$ for $\delta = 10^{-5}$.

Q14: How does the global federated model perform compared to the centralized model?

A: The federated global model converges by Round 5 to a test MAE of 2.920 cycles, closely matching the centralized baseline of 2.911 cycles.

Q15: What framework coordinates federated execution in production?

A: The Flower (flwr) federated learning framework using gRPC transport layers for communication between edge clients and the central coordinator.

Q16: How does TreeSHAP achieve polynomial time complexity?

A: TreeSHAP evaluates conditional expectations recursively across tree nodes, reducing computational complexity from exponential $O(2^M)$ to polynomial $O(T * L * D^2)$.

Q17: How does ForgeSight AI compute TreeSHAP values without sharing raw background data?

A: It uses the StandardScaler mean vector as a summary background baseline, calculating local attributions without exposing raw client datasets.

Q18: How does the counterfactual engine generate actionable recommendations?

A: It optimizes an objective function using the L-BFGS-B algorithm, finding minimal feature modifications that adjust predicted RUL to a target state within physical bounds.

Q19: What is Split Conformal Prediction, and what calibration quantile was obtained?

A: Split conformal prediction is a distribution-free method for bounding point predictions. Evaluated on 4,392 calibration samples, it yields an uncertainty threshold of $q_{\hat{}} = 5.01$ cycles at a 95% significance level.

Q20: Why does empirical test coverage (79.8%) fall short of the nominal 95% target?

A: Due to a distribution shift between complete run-to-failure training sequences and right-censored test sequences, which violates the exchangeability assumption of conformal prediction.

Q21: What technologies power the 3D Digital Twin?

A: React 18, TypeScript, Three.js, and React-Three-Fiber (R3F), utilizing WebGL for client-side 3D rendering.

Q22: At what frame rate does the 3D Digital Twin render?

A: At 60.0 FPS with an active rendering memory footprint of approximately 85 MB RAM.

Q23: What is the real-time WebSocket synchronization latency?

A: The WebSocket synchronization delay averages 4.8 ms, measured using Chrome Developer Tools over 1,000 rendering frames.

Q24: How are component health scores mapped to 3D materials?

A: Component health scores (0.0 to 1.0) update PBR material colors dynamically: Emerald Green for healthy (> 0.70), Amber Yellow for degraded ($0.30 - 0.70$), and Ruby Red for critical (< 0.30).

Q25: How does the backend push live predictions to the digital twin?

A: Via an asynchronous WebSocket server route (`app/api/streaming.py`) that serializes prediction dictionaries into JSON payloads dispatched to connected clients.

Q26: Why use the Singleton pattern for the Model Registry?

A: To ensure model weights (joblib files) are loaded into RAM only once at application startup, avoiding file I/O overhead and memory leaks during inference calls.

Q27: How are API payloads validated in FastAPI?

A: Using Pydantic data schemas (`app/domain/schemas.py`), enforcing strict type checking and serialization for incoming JSON objects.

Q28: How does the multi-method anomaly engine work?

A: It combines Isolation Forest, Local Outlier Factor (LOF), and Z-score metrics to identify out-of-distribution telemetry before it reaches the prognostic models.

Q29: How is the application deployed in containerized environments?

A: Containerized using Docker (Dockerfile) and orchestrated via `docker-compose.yml` or Kubernetes manifests (`deployment.yaml`) with readiness and liveness probes.

Q30: Is ForgeSight AI restricted to turbofan engine telemetry?

A: No. While evaluated on NASA C-MAPSS turbofan data, the underlying prognostic framework, conformal calibration, TreeSHAP explanations, and 3D digital twins are machine-agnostic and can be applied to other industrial assets by updating sensor schemas.

Q31: Derive the optimal leaf weight formula in XGBoost.

A: Setting $\partial L / \partial w_j = 0$ yields $w_j^* = -G_j / (H_j + \lambda)$.

Q32: What is the mathematical definition of a Shapley value?

A: $\phi_j(x) = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(|N|-|S|-1)!}{(|N|-1)!} [f(S \cup \{j\}) - f(S)]$.

Q33: How does Rényi Differential Privacy (RDP) differ from standard (epsilon, delta)-DP?

A: RDP uses Rényi divergence to measure privacy loss, providing tighter composition bounds for Gaussian noise across multiple training rounds.

Q34: What is the exchangeability assumption in Conformal Prediction?

A: It assumes that calibration and test data samples are exchangeable (i.e. their joint probability distribution is invariant to permutations).

Q35: How does importance-weighted conformal prediction resolve distribution shifts?

A: It reweights calibration residuals using the likelihood density ratio $w(x) = p_{\text{test}}(x) / p_{\text{train}}(x)$, restoring valid coverage under distribution shifts.

Q36: What is the computational complexity of TreeSHAP execution?

A: $O(T * L * D^2)$, where T is trees, L is leaves, and D is max depth.

Q37: How does L-BFGS-B optimize the counterfactual objective?

A: Limited-memory BFGS with Bounds (L-BFGS-B) uses quasi-Newton gradient evaluations to minimize losses subject to upper and lower feature bounds.

Q38: What role does StandardScaler play in feature engineering?

A: It standardizes features to zero mean and unit variance $z = (x - \mu) / \sigma$, preventing features with large raw magnitudes from dominating model split evaluations.

Q39: What is the significance of the R^2 score metric in regression evaluation?

A: $R^2 = 1 - \text{SSE} / \text{SST}$. It measures the proportion of variance in the target variable explained by the model (0.957 for our XGBoost model).

Q40: What happens if a WebSocket connection drops during streaming?

A: The custom React hook useWebSocket.ts detects the connection drop and attempts exponential backoff reconnection while displaying a visual status banner on the UI.

Q41: How are sliding windows constructed during real-time streaming?

A: Incoming telemetry rows are pushed into an in-memory deque buffer per unit ID. When the buffer reaches window size w, rolling statistics are calculated.

Q42: What is the role of the lifespan.py module in FastAPI?

A: It defines async context manager functions that execute code on server startup (initializing ModelRegistry) and shutdown (releasing file handles and buffers).

Q43: How does the system prevent feature leakage during scaling?

A: The StandardScaler is fitted strictly on the training set split. Test and calibration sets are transformed using these pre-computed training metrics.

Q44: Why use exponential moving averages (EMA) alongside rolling means?

A: Rolling means weight all values in a window equally, whereas EMAs apply exponentially decaying weights to older observations, making them more responsive to recent trend shifts.

Q45: What is the total line count and layout format of the paper manuscript?

A: The paper comprises 766 LaTeX lines formatted under the two-column `ieeconf` IEEE conference template.

Q46: How are multiple client models stored during federated training?

A: The Flower coordinator stores client tree parameters in dictionary arrays, appending structural node definitions into the global ensemble model.

Q47: What visual feedback indicates an anomalous sensor input?

A: The `AnomalyAlertBanner.tsx` component displays a prominent alert overlay on the dashboard, and the affected component highlights in high-contrast red on the 3D twin.

Q48: How does the system handle missing telemetry inputs?

A: Missing input fields are imputed using pre-computed feature median values from the training split, maintaining model stability.

Q49: What is the memory footprint of the backend server during idle operation?

A: Approximately 45.0 MB RAM idle, expanding to 85.0 MB during active WebGL streaming rendering.

Q50: What are the primary directions for future work?

A: Porting federated edge nodes to physical hardware (Raspberry Pi/Jetson), implementing online continual learning under drift, and ingesting multimodal sensor data (vibration waveforms, thermal imaging).